

Null Space Networks and Adversarial Attacks in Limited-Angle Computed Tomography

Noah

June 2026

Contents

1	Introduction	2
1.1	Limited-angle CT	2
1.2	Pseudoinverse	4
1.3	Residual UNet	6
1.3.1	Neural Networks	6
1.3.2	Residual networks	7
1.3.3	U-Net	7
1.4	Null space Network	8
1.5	Adversarial Attacks	9
2	Theory behind Adversarial Attacks	10
2.1	Adversarial Attacks	10
2.1.1	Targeted Adversarial Attacks on Subspaces	11
2.1.2	Targeted Adversarial Attacks using Examples	12
3	Numerical implementation	12
3.1	Radon operator	14
3.2	Pseudoinverse	15
3.3	Data generation	17
3.4	Residual UNet	19
3.5	Nullspace Network	22
3.6	Adversarial attacks	24
3.6.1	Targeted Adversarial Attacks on Subspaces	26
3.6.2	Targeted Adversarial Attacks using Examples	27
3.6.3	Epoch study	27
3.7	Metrics and error decomposition	28
3.8	Experimental pipeline	31

4	Results	32
4.1	Model performance	32
4.1.1	Reconstruction accuracy	32
4.1.2	Dataconsistency	32
4.2	General adversarial attack	32
4.3	Targeted adversarial attack	32
4.4	Epoch study	32
5	Conclusion and Outlook	32
5.1	Conclusion	32
5.2	Outlook	32

1 Introduction

Computed Tomography is an important diagnosis tool in modern medicine, it can be used to identify various injuries or diseases inside the body. Examples include screening the chest for viral and bacterial pneumonia, tuberculosis, lung cancer, the bones and teeth for fractures and infections and many more[1]. Reconstructing the image of the tissue from the measurements poses an inverse problem that is usually solved using the filtered back projection. This reconstruction, however, produces artifacts and unsatisfactory image quality [2], when applied to measurements resulting from a Limited Angle CT-Scan. Due to the severe Ill-posedness of the Limited Angle CT-Scan, an accurate reconstruction method for the Limited Angle setting is yet to be found. Reconstructions using neural networks form a promising approach, as neural networks have been used successfully in different kinds of inverse problems. One big disadvantage of such reconstruction methods, however, is their vulnerability towards deliberate manipulations of the input [3]. Since CT-Scans are used all over the world, such vulnerable reconstruction methods would pose a risk in patient safety. That is, why this thesis investigates the robustness of the reconstruction method. Before we get into the details of how the reconstruction method works, and how we plan to attack it, we first need to understand, how computed Tomography and Limited Angle Computed Tomography in particular actually work.

1.1 Limited-angle CT

To understand how a CT-scan works, let us first look at general X-ray imaging. Sending an x-ray through a body tissue and measuring its intensity at the other side of the body is a way of measuring the average density of the tissue the ray was sent through. By arranging x-ray emitters and detectors in a plane and putting the tissue one wants to investigate in between them, an x-ray image can be produced.

CT uses the same mechanism of tissue absorbing x-rays based on the tissues density. There are two main ways of doing CT: fan-beam and parallel-beam. Since fan-beam is the more complex one, in this work we restrict ourselves to

parallel-beam CT and whenever CT will be mentioned in this work, we are talking about parallel-beam CT. In CT one uses emitters and detectors lying in the slice plane perpendicular to the rotation axis. Rotating this array of x-ray sources and detectors around the tissue generates different measurements for each measured angle; Figure 1 shows the resulting parallel-beam geometry. Usually one rotates the full 180° to not miss any features. These measurements can be composed into one 2D Image called a sinogram. Each row of a sinogram corresponds to one angle, each column to one detector. Such a sinogram can then be used to reconstruct the axial image or slice.

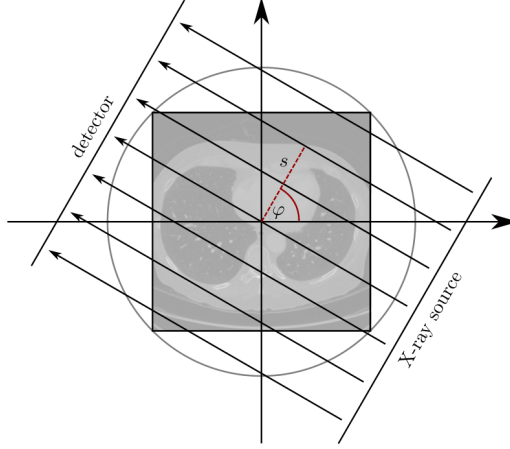


Figure 1: Parallel-beam geometry. For a fixed projection angle the source emits a bundle of parallel x-rays through the object and the opposing detector array records their attenuation. A single ray is identified by the angle φ (written θ in the text) and the signed offset s from the center of rotation. Rotating the source-detector pair through a range of angles and stacking the resulting profiles produces the sinogram.

The number of measurements required during a CT-scan depends on the number of angles that one measures and the amount of detectors used during the scan. More measurements mean longer measuring process and also increased exposure of the tissue to radiation.

Consequently research on ways to reduce the amount of measurements, while still producing sufficiently accurate reconstructions is of both economic and medicinal interest. One way to reduce the amount of measurements is to reduce the angles in total while retaining the total coverage of angles, i.e. measuring in 2° instead of 1° steps, reducing the total amount of measurements by one half.

Another way, which will also be the focus of this work, relies on the reduction of the total angles covered. Instead of considering measurements across the full range from 0° to 180° one only measures from 0° to 120° . This is called limited-angle CT. It is severely ill-posed and methods like the filtered back projection,

which work well for conventional CT-scans, fail to provide good reconstruction images [2].

The reconstruction method used in this thesis is not only based on neural networks, but also on a classical reconstruction step. Classical reconstruction methods better understood and provide a good initial construction, which serves as an useful input to the neural network. The network therefore only needs to learn the negative error of the classical reconstruction method to the ground truth.

1.2 Pseudoinverse

The Radon transform is the function, that can be used to model the x-ray measurement process. It builds the line integral of a function and is defined in the following way.

$$\mathcal{R}f(L) := \int_L f(\mathbf{x})d\mathbf{x}$$

Where R denotes the radon transform, $f : \Omega \rightarrow \mathbb{R}$ is an arbitrary, measurable function and $L \subset \Omega$ the line along which the measurement is taken. Since in Computed Tomography the lines are strictly defined by an offset s and an angle θ , we can also write this in the following way:

$$\mathcal{R}f(\theta, s) := \int_{\mathbb{R}} f(s\omega_{\theta} + t\omega_{\theta}^{\perp})dt, \quad \omega_{\theta} = (\cos \theta, \sin \theta)$$

It is easy to see, that this is a linear operator. Under the assumption, that the Ω is expressible as a finite pixel-basis and that we only make $m \in \mathbb{N}$ measurements $(\theta_i, s_i) \quad i \in \{1, \dots, m\}$, we can use a Matrix to model the x-ray measurement process. Assigning each pixel a number from 1 to n allows us to define our function $f(x) = \sum_j c_j \varphi_j(x) = \sum_j c_j \mathbf{1}_j(x) \quad x \in \{1, \dots, n\}$ as a finite sum of weighted basis functions. These weights can be grouped into one vector $c \in \mathbb{R}^n$. Now applying the Radon transform and using linearity gives us for the measurement y_i corresponding to (θ_i, s_i) the following equation.

$$y_i = \mathcal{R}f(\theta_i, s_i) = \sum_j c_j \mathcal{R}\varphi_j(\theta_i, s_i) = \sum_j c_j \mathbf{A}_{ij} = e_i^T \mathbf{A} c$$

Aggregating every measurement into one row we receive the linear equation with $\mathbf{A}$ defined by $\mathbf{A}_{ij} := \mathcal{R}\varphi_j(\theta_i, s_i)$.

$$y = \mathbf{A} c$$

Since the parameters of the radon transform θ and s along with the choice of basis functions uniquely define $\mathbf{A} \in \mathbb{R}^{m \times n}$ we write it as $\mathbf{A}_{\mathcal{R}}$.

This simplifies the implementation and allows us to use the pseudoinverse $A_{\mathcal{R}}^+$ to get a reconstruction of the original image. Due to a large requirement of memory and computational power, this approach is not being used in clinical CT. Images in clinical CT require 512×512 pixels and low-dose clinical CT

works with 700 – 900 detector elements and about 1000 projections per 360° slice, which corresponds to 500 projections per 180°. In this way we obtain a Matrix of size 350 000 × 262 144 [4]. Singular Value Decomposition for matrices in this size, take a long time and are therefore not feasible in clinical CT. In this laboratory setting however, our matrix is smaller and using it, simplifies concepts like null space and its complement. This will also be of importance, when we talk about null space networks and error decomposition in null space and null space complement.

The construction of the Pseudoinverse is done using the singular value decomposition. It is a useful tool that generalizes the eigenvalue decomposition to non square matrices. A general Matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ can be written in the following way:

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^*$$

Where $\mathbf{U} \in \mathbb{R}^{n \times n}$ and $\mathbf{V} \in \mathbb{R}^{m \times m}$ are unitary matrices and $\mathbf{\Sigma} \in \mathbb{R}^{n \times m}$ is a diagonal matrix with r non negative real numbers σ_i on the diagonal, where r is the rank of the matrix $\mathbf{A}$. We fix the ordering once and use it for the rest of this work. The singular values are sorted in descending order,

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0,$$

and the columns of $\mathbf{U}$ and of $\mathbf{V}$ are ordered to match, so that the i -th column of each belongs to σ_i . Two things depend on this convention. $\sigma_1 = \sigma_{\max} = \|\mathbf{A}\|_2$ is the operator norm, which is what the truncation threshold of the next paragraph is measured against; and “the first k columns of $\mathbf{V}$ ” is then an unambiguous description of the vectors a truncation keeps. The decomposition returned by `torch.linalg.svd` is already sorted this way. The Pseudoinverse of $\mathbf{A}$ is then easily constructed.

$$\mathbf{A}^+ = \mathbf{V}\mathbf{\Sigma}^+\mathbf{U}^* = \mathbf{V} \left(\begin{array}{c|c} \begin{array}{ccc} \frac{1}{\sigma_1} & & \\ & \ddots & \\ & & \frac{1}{\sigma_r} \end{array} & \begin{array}{c} 0 \\ \\ 0 \end{array} \\ \hline \begin{array}{c} 0 \end{array} & \begin{array}{c} 0 \end{array} \end{array} \right) \mathbf{U}^*$$

This pseudoinverse can then be used to convert a sinogram back to the axial image. Since in real world application sinogram data always comes with noise, To better represent real world applications we are also going to need to add some noise to the data, as real world measurements can never be noise free. But Noisy data reveals an issue with the pseudoinverse. The severe ill-posedness means, singular values σ_i decay exponentially to zero. For the Pseudoinverse, this means its singular values $1/\sigma_i$ grow exponentially. Since noise is spread evenly across the singular values of the pseudoinverse, it is amplified exponentially and can overshadow the actual signal strength. To counteract this, one can

use a truncated pseudoinverse, setting all singular values σ_i , smaller than some threshold τ , to zero. When inverting, they remain zero and the amplification can be bounded with $1/\tau$.

This gives rise to a nontrivial null space $\mathcal{N}(\mathbf{A})$ i.e. non trivial structures, that when measured give 0 measurements. This allows us to decompose the space $\mathbb{R}^n$ into $\mathcal{N}(\mathbf{A})$ and $\mathcal{N}(\mathbf{A})^\perp$ which will be of use later in this work.

To avoid any confusion, we fix the notation we are going to use throughout this work now.

symbol	definition	what it is
$\mathbf{A}$		limited-angle system matrix
$\mathbf{A}_{\text{full}}$		system matrix of <i>all</i> angles
$\mathbf{M}$		0/1 mask over the measured rows
τ		relative truncation threshold
$\mathbf{U}_k, \Sigma_k, \mathbf{V}_k$		SVD factors of $\mathbf{A}$ retained at τ
$\mathbf{A}^+$	$\mathbf{V}_k \Sigma_k^{-1} \mathbf{U}_k^*$	truncated pseudoinverse
$\mathbf{P}_k$	$\mathbf{U}_k \mathbf{U}_k^*$	sinogram-space projector
$\mathbf{P}_{\mathcal{N}(\mathbf{A})}$	$\mathbf{I} - \mathbf{V}_k \mathbf{V}_k^*$	image-space projector
$\mathbf{Y}$		space of sinograms or measurements
$\mathbf{X}$		space of images

$\mathbf{A}_{\text{full}}$ appears only once, in Section 3.1, when we construct $\mathbf{A}$ from it by deleting the rows of the unmeasured angles. Everywhere else the operator we are going to use is $\mathbf{A}$.

1.3 Residual UNet

Before we get into the key property of a residual network, we need to understand how a general neural network is built.

1.3.1 Neural Networks

A neural network consists of connected neurons, which are usually grouped into layers. Each neuron has multiple weighted inputs. Each input additionally is given a bias. All inputs are then combined and given to an activation function whose output is the output of the neuron. The activation function is usually nonlinear. One neuron can therefore be modeled in the following way:

$$\mathbf{y} = \phi\left(\sum_{i=1}^n w_i \mathbf{x}_i + b\right) \quad w_i, \mathbf{x}_i, b \in \mathbb{R}$$

Where $\mathbf{y}$ is the output, $\phi : \mathbb{R} \rightarrow \mathbb{R}$ is the activation function, $w_i \mathbf{x}_i$ are the weighted inputs and b is the offset. The layers of the neural network are executed in sequence, the input layer receives the network input and feeds its output into the next layer. The connections between the different layers are an architectural choice. Fully Connected Neural Networks feed the output of

one neuron into every neuron of the next layer. A different approach, the so called Convolutional Neural Network, can be compared to the convolution with a small kernel. To understand it, it is best to think of the layers of a neural network as a grid. We can then talk about neurons of one layer neighboring neurons of the next layer. A convolutional neural network feeds the output of one neuron only into neighboring neurons of the next layer. According to these connections, iteratively each hidden layer is computed until the data arrives at the output layer and produces the network output.

1.3.2 Residual networks

Up to some threshold, deeper models perform better than shallow ones [5]. Nevertheless, it has been reported, that adding more layers to a suitably deep model, increases its training error [6]. This is surprising, as it is easy to find a trivial deep model, performing on par with the shallow model. It can be constructed by supplementing the shallow model with layers that are the identity. Such a model possesses a larger depth, but has equivalent performance as the shallow, used to construct it. This indicates, that some deep networks are harder to optimize, than shallow ones. Since residual networks have been introduced in 2015, they have become a popular Convolutional Neural Network architecture. They address this degradation problem by learning the residual instead of the whole mapping. Let $\mathcal{H} : \mathbf{Y} \rightarrow \mathbf{X}$ be the true mapping from measurements to images and $\mathcal{I} : \mathbf{Y} \rightarrow \mathbf{X}$ be the identity, if measurements and images live in the same space and otherwise an easy to calculate classical reconstruction map. The residual mapping learned by the neural network is then given by $\mathcal{G} := \mathcal{H} - \mathcal{I}$ [7]. In our setting the two spaces differ, so for $\mathcal{I}$, the classical reconstruction map, we choose the truncated pseudoinverse $\mathbf{A}^+$. The residual is learned by a network $N_\theta : \mathbf{X} \rightarrow \mathbf{X}$ acting on the initial reconstruction. Writing $\mathbf{x}_{\text{init}} := \mathbf{A}^+ \mathbf{y}$, the full reconstruction map is

$$\mathcal{F}_\theta : \mathbf{Y} \rightarrow \mathbf{X}, \quad \mathcal{F}_\theta(\mathbf{y}) = \mathbf{x}_{\text{init}} + N_\theta(\mathbf{x}_{\text{init}}).$$

One might notice, that the residual mapping we use is distinct from the definition above. Instead of defining the network from $\mathbf{Y}$ to $\mathbf{X}$ we define it from $\mathbf{X}$ to $\mathbf{X}$ by concatenating the network to the truncated pseudoinverse $\mathbf{A}^+$. We refer to N_θ as the network and to $\mathcal{F}_\theta$ as the reconstruction method. Both symbols are used with these meanings for the remainder of this work.

1.3.3 U-Net

So far we have only talked about the general architecture of the model. Now we also need to address the architecture of the neural network itself. The architecture we are going to use is the U-Net. It is a special type of convolutional neural network, that maps images to images. The UNet can be segmented into two paths, one contracting and one expansive path. The contracting path is

built on the repeated application of convolutions, each followed by a rectified linear unit (ReLU) and a max pooling operation for down sampling [8].

As mentioned before a convolutional neural network restricts the neurons in the next layer, that receive the output of a neuron in the current layer. A convolution with a kernel of size 3×3 can only influence the neuron corresponding to itself and its surrounding 8 neighbors. The ReLU function is a popular activation function. It is the identity for positive input values and maps everything else to 0. The max pooling operation splits the output of a layer into different pools and combines them into one output by taking the largest value in the pool. If for instance you have a 4×4 layer, max pooling outputs values in a 3×3 or 2×2 grid depending on the application, which is influenced by the stride parameter.

$$\left(\begin{array}{cc|cc} \sigma_1 & \sigma_2 & \sigma_3 & \sigma_4 \\ \sigma_5 & \sigma_6 & \sigma_7 & \sigma_8 \\ \hline \sigma_9 & \sigma_{10} & \sigma_{11} & \sigma_{12} \\ \sigma_{13} & \sigma_{14} & \sigma_{15} & \sigma_{16} \end{array} \right) \left(\begin{array}{cc|cc} \sigma_1 & \sigma_2 & \sigma_3 & \sigma_4 \\ \sigma_5 & \sigma_6 & \sigma_7 & \sigma_8 \\ \sigma_9 & \sigma_{10} & \sigma_{11} & \sigma_{12} \\ \sigma_{13} & \sigma_{14} & \sigma_{15} & \sigma_{16} \end{array} \right)$$

This figure illustrates the different applications, the red highlighted matrix outputs a 2×2 grid of values, since it is moved with stride 2 in between steps and therefore does not take any values twice. The blue one outputs a 3×3 grid. It only moves one value (stride 1) in between steps and therefore takes some values multiple times. The U-Net uses stride 2, which leads to a reduction in the number of neurons and weights by the factor 4. The memory gained by reducing neurons and weights is used to introduce new feature channels. Feature channels are separate data channels, that can be used to The 2×2 max pooling halves the spatial resolution but leaves the number of channels unchanged. The number of feature channels is then doubled by the first convolution of the following block, which uses twice as many filters as that block has input channels. After the contractive path, the expansive path applies a form of upsampling with another 2×2 convolution, halving the number of feature channels. The upsampled data is then combined with some data from the corresponding down-sampling step and twice more convolved, followed by a ReLU application. The original U-Net Network has 23 convolutional layers[8]. Its original application has been for segmentation purposes, however it is also applicable for image to image prediction.

1.4 Null space Network

The Residual U-Net is a so called unconstrained Network, it can modify the data, however it desires, to predict its output. This can lead to problems, since it can produce predictions, that do not agree with the measurements it was fed. One possible remedy to this issue is the Null Space Network. It's architecture is identical to the Residual U-Net with the exception, that the output of the Null Space Network is Projected onto the Null Space of the Forward Operator (i.e. the measurement process). This guarantees data consistency and we had hoped, that consequently the resulting network would be more robust towards

adversarial attacks. Adversarial attacks on both residual UNet and Null space networks however, have shown, that this is not the case and that resistance towards adversarial attacks or robustness, do not come for free with data consistency.

To use a Null Space Network, one has to know a Projection onto the Null Space of the Forward Operator or Measurement Process. This is where the Singular Value Decomposition comes in useful once more, the unitary matrix $\mathbf{V}$ can be used to calculate the Nullspace projection in the following way. With the descending ordering of Section 1.2, the k directions that are kept are the *first* k columns of $\mathbf{V}$, and collecting them in the matrix $\mathbf{V}_k$ allows us to calculate the Projection $\mathbf{P}_{\mathcal{N}(\mathbf{A})}$ by

$$\mathbf{P}_{\mathcal{N}(\mathbf{A})} = \mathbf{I} - \mathbf{V}_k \mathbf{V}_k^*$$

This allows us to express the Nullspace Network as a concatenation of the projection $\mathbf{P}_{\mathcal{N}(\mathbf{A})}$ and the network, where $\mathbf{x}_{\text{init}}$ is an initial reconstruction obtained by some classical reconstruction method, in our case the Pseudoinverse $\mathbf{A}^+$.

$$\mathbf{x}_{\text{out}} = \mathcal{F}_\theta(\mathbf{y}) = \mathbf{x}_{\text{init}} + \mathbf{P}_{\mathcal{N}(\mathbf{A})} N_\theta(\mathbf{x}_{\text{init}})$$

By Applying $\mathbf{A}$ to the above equation, it is easy to see, that the Network output can not modify the measurements gained from the initial reconstruction.

$$\mathbf{A} \mathbf{P}_{\mathcal{N}(\mathbf{A})} = 0 \implies \mathbf{A} \mathbf{x}_{\text{out}} = \mathbf{A} \mathbf{x}_{\text{init}}$$

The Network is provably data consistent. This means measuring an image produced by the nullspace network will always yield the exact measurements used for that prediction. Consequently though, any errors in the input it has been given, that produce non zero measurements, can never be corrected by the network.

Considering that some noise will always be visible in the measurements, even after the application of the pseudoinverse, and that this Noise alone will produce non zero measurements, the error generated by this noise is not correctable by the Nullspace Network. Decomposing the reconstruction error $\mathbf{e} := \mathbf{x}_{\text{out}} - \mathbf{x}^\dagger$ into its nullspace and nullspace-complement components makes this precise. The nullspace-complement component $(\mathbf{I} - \mathbf{P}_{\mathcal{N}(\mathbf{A})})\mathbf{e}$ of the reconstruction error is identical to that of the initial reconstruction, whatever the network predicts. We will refer to this as the *range floor*. In the nullspace-complement the Nullspace Network can therefore never produce results that are comparable to those of the unconstrained residual network, which is free to correct this component and does.

The projection also gives us a decomposition of the space of reconstructions into the null space $\mathcal{N}$ and its complement $\mathcal{N}^\perp$. This will be used not only to analyze our attacks later and to construct a new type of attack, used in this work.

1.5 Adversarial Attacks

An adversarial attack is the targeted construction of a small perturbation to a specific input to a model, that maximizes the error of the reconstruction. Depending on how this error is measured and how much the threat model knows

about the model they can differ significantly [9]. In this work we want to use adversarial attacks to investigate the robustness of the residual UNet and the Null space network. The next section describes adversarial attacks in more detail and explains modifications we made to the general attack framework to better evaluate our models.

2 Theory behind Adversarial Attacks

Let $\mathcal{F}_\theta : \mathbf{Y} \rightarrow \mathbf{X}$ denote any reconstruction method going from the space of measurements to the space of images. In the following section we are going to talk about different methods to attack $\mathcal{F}_\theta$ in different ways.

2.1 Adversarial Attacks

An adversarial attack is heavily dependent on the threat model. It defines how much information the attack is given and how we measure its success. The attack type that we are going to investigate in this work is a so called white-box attack. Unlike black-box attacks, a white-box attacker has unlimited knowledge of the reconstruction method. In our case that means, the attacker is aware of the exact model architecture as well as of its weights. Such an attack is of course a lot more powerful, than a black-box attack, where the attacker has no knowledge of the reconstruction method. As mentioned before, the aim is to find a small perturbation $\delta^* \in \mathbf{Y}$ defined by:

$$\delta^* = \arg \max_{\|\delta\|_p \leq \varepsilon} \mathcal{L}(\mathcal{F}_\theta(\mathbf{y} + \delta), \mathbf{x}^\dagger)$$

where $\mathbf{y}$ are the measurements, $\mathbf{x}^\dagger$ is the ground truth corresponding to them, $\mathcal{L}$ is a loss function and $\varepsilon \in \mathbb{R}_{>0}$ is the budget of the attack. Adversarial attacks can use different strategies to find such a perturbation. In this work we relied on the projected gradient descent optimizer (PGD), Madry et al. [10] claim this to be the strongest attack utilizing the local first order information of the network.

Projected gradient descent is a general method for minimising a differentiable function over a closed convex set: one takes a gradient step and projects the result back onto the set. Suppose we want to attack the groundtruth, sinogram pair $(\mathbf{x}^\dagger, \mathbf{y})$. For this attack the closed convex set is the ball of radius ε around the clean sinogram,

$$\mathbf{y} + B_\varepsilon := \{\mathbf{z} \in \mathbf{Y} : \|\mathbf{z} - \mathbf{y}\|_2 \leq \varepsilon\},$$

this ensures that every iterate stays within the budget of the attack. A PGD attack is started from a random point $\mathbf{y}_0 \in \mathbf{y} + B_\varepsilon$. Each step is then calculated as follows:

$$\mathbf{y}_{n+1} := \Pi_{\mathbf{y} + B_\varepsilon}(\mathbf{y}_n - \alpha \mathbf{g}_n), \quad \mathbf{g}_n := \nabla_{\mathbf{z}} \mathcal{L}(\mathcal{F}_\theta(\mathbf{z}), \mathbf{x}^\dagger) \Big|_{\mathbf{z}=\mathbf{y}_n}$$

where $\alpha \in \mathbb{R}_{\geq 0}$ is the step size, $\Pi_{\mathbf{y}+B_\varepsilon}$ is the projection onto that ball and $\mathcal{L}$ is the same loss function as above. Note that the ball is centred at $\mathbf{y}$ and not at the origin: we are looking for a small perturbation of one specific sinogram, not for a sinogram of small norm.

Two changes turn this minimiser into the attack we use. First, we maximise instead of minimise, so the gradient step is added rather than subtracted. Second, the raw gradient is replaced by the direction of steepest ascent with respect to the norm in which the budget is measured. For a step of length α that direction maximises $\langle \mathbf{g}, \mathbf{d} \rangle$ over $\|\mathbf{d}\| \leq \alpha$, which for the two norms of interest gives

$$\ell_\infty : \quad \alpha \operatorname{sgn}(\mathbf{g}), \quad \ell_2 : \quad \alpha \frac{\mathbf{g}}{\|\mathbf{g}\|_2}.$$

Madry et al. [10] state the algorithm with the sign function, because their threat model is bounded in the ℓ_∞ norm. Our budget is measured in the ℓ_2 norm, so we use the normalised gradient instead, and the step rule and the constraint set are then expressed in the same norm.

A last adaption concerns the feasible set. We investigate only those perturbations of the measurements that an attacker could realistically produce, that is, perturbations which are themselves measurements taken at the angles the scanner actually covers. We therefore additionally require the perturbation to lie in $\operatorname{range}(\mathbf{A})$, which gives the feasible set $S := \operatorname{range}(\mathbf{A}) \cap B_\varepsilon$. If we write the iteration in terms of the perturbation $\delta_n := \mathbf{y}_n - \mathbf{y}$ instead of the sinogram itself, which translates both the iterate and the constraint set and therefore describes the same algorithm, the update rule becomes

$$\delta_{n+1} := \Pi_S \left(\delta_n + \alpha \frac{\mathbf{g}_n}{\|\mathbf{g}_n\|_2} \right), \quad \mathbf{g}_n := \nabla_\delta \mathcal{L}(\mathcal{F}_\theta(\mathbf{y} + \delta), \mathbf{x}^\dagger) \Big|_{\delta=\delta_n}$$

with the starting point δ_0 drawn at random from S . This is the blueprint for every attack we used to test the robustness of our models. The different attacks we used describe only different choices of the loss $\mathcal{L}$.

2.1.1 Targeted Adversarial Attacks on Subspaces

With the blueprint we have just given, it is already possible to perform successful attacks on both models. During our work, however, we noticed that when attacking a Nullspace Network with the plain squared-error loss, the error found by the attack grew almost entirely in the nullspace complement. By the range floor of Section 1.4 this is precisely the component the network does not control: it is inherited unchanged from the initial reconstruction, so an attack that grows it is exploiting the architecture rather than the learned reconstruction. The resulting error is large, but it is not evidence of a vulnerability, and it is not comparable to what the same attack achieves against the unconstrained residual network, which is free to correct that component.

We addressed this issue by changing the loss function $\mathcal{L}$ we use in the attack.

Let

$$\mathbf{e}(\boldsymbol{\delta}) := \mathcal{F}_\theta(\mathbf{y} + \boldsymbol{\delta}) - \mathbf{x}^\dagger$$

denote the reconstruction error. Instead of its total energy we maximise the energy of a single component of its decomposition into the two subspaces $\mathcal{N}_\mathbf{A}$ and $\mathcal{N}_\mathbf{A}^\perp$,

$$\mathcal{L}_{\mathcal{N}_\mathbf{A}} := \|\mathbf{P}_{\mathcal{N}(\mathbf{A})} \mathbf{e}(\boldsymbol{\delta})\|_2^2, \quad \mathcal{L}_{\mathcal{N}_\mathbf{A}^\perp} := \|(\mathbf{I} - \mathbf{P}_{\mathcal{N}(\mathbf{A})}) \mathbf{e}(\boldsymbol{\delta})\|_2^2.$$

Both projections are linear and differentiable, so the attack of the previous section applies unchanged; only the loss whose gradient is taken is replaced. Maximising $\mathcal{L}_\mathcal{N}$ forces the attack to damage the nullspace component, which is the only part of the reconstruction subject to the influence of the Nullspace Network. Since the residual network is unconstrained, we can compare the robustness of the two different networks better by constraining the attack to the nullspace component. Maximising $\mathcal{L}_{\mathcal{N}^\perp}$ gives the complementary attack and serves as a control. For the Nullspace Network, it can only ever attack the Pseudoinverse. For the Residual UNet, it shows how far the measurements of the reconstruction can be moved by a small perturbation.

2.1.2 Targeted Adversarial Attacks using Examples

Another modification to our attack is made by altering the attack objective so that it no longer drives the reconstruction *away* from the ground truth, but *towards* a prescribed image. Attacks of this kind are called targeted. Let $\mathbf{t} \in \mathbf{X}$ be a target image and replace the loss by

$$\mathcal{L}_\mathbf{t} := -\|\mathcal{F}_\theta(\mathbf{y} + \boldsymbol{\delta}) - \mathbf{t}\|_2^2.$$

Since the attack maximises $\mathcal{L}$, maximising $\mathcal{L}_\mathbf{t}$ minimises the distance between the reconstruction and the target, and the update rule of the previous section can again be used unchanged.

We use two choices of target. The first is the zero image, $\mathbf{t} = 0$, which asks how far towards an empty reconstruction an attacker can push the model within the budget. The second is the ground truth of a *different* sample, so that the attack attempts to make the reconstruction of one object resemble another. It asks not whether the reconstruction can be corrupted, but whether it can be corrupted into something that still looks like a plausible scan of a different patient. A targeted attack is a strictly harder problem than an untargeted one, because the attacker must reach one particular point of image space rather than any point far from the ground truth, and we should expect correspondingly smaller errors at the same budget.

3 Numerical implementation

This chapter describes how the operators, networks and attacks of the previous two chapters are realised in code. The implementation is written in Python

and rests on two libraries: PyTorch [11] provides the tensor algebra, the neural network layers and, most importantly for this work, reverse-mode automatic differentiation, and the ASTRA Toolbox [12] provides the tomographic geometry. Every number reported in this thesis was produced with PyTorch 2.3.1 on a single NVIDIA RTX A6000.

The central object of the implementation is the *tensor*: a multidimensional array that carries a shape, a numeric type and the device it lives on (CPU or GPU), and that optionally records the operations which produced it. That record is the computational graph, and it is what lets PyTorch differentiate a value with respect to any tensor it was computed from. Two shapes occur throughout. An image is a tensor of shape (B, C, H, W) — batch, channel, height, width — and a sinogram is a tensor of shape (B, C, n_θ, n_d) — batch, channel, projection angle, detector element. Here B is the number of samples processed simultaneously and $C = 1$, since our images are greyscale. Every operator described below maps between these two shapes and acts on a whole batch in a single call.

One convention deserves to be stated in advance, because it recurs in every later section. The limited-angle operators do not return a shortened sinogram containing only the measured rows; they return a *full-shape* sinogram in which the rows belonging to unmeasured angles are set to zero. Every sinogram in a run therefore has the same shape, whether it was produced by the full or by the limited-angle operator, and the restriction to the measured angles is itself an operator, namely the multiplication with a 0/1 mask, we denote this mask in the following with $\mathbf{M}$. $\mathbf{M}$ is a tensor of shape (n_θ, n_d) with 1 in the rows corresponding to measured angles and 0 elsewhere.

It is important to note that $\mathbf{M}$ is not the same thing as $\mathbf{P}_k$, the projector onto the retained range. Zeroing the unmeasured rows says only that a sinogram is supported on the angles the scanner covers; it does not say that some image would produce it. The sinograms that are measurements of an image form $\text{range}(\mathbf{A})$, a proper subspace of $\text{range}(\mathbf{M})$, and those of *them* that survive the truncation form $\text{range}(\mathbf{U}_k)$, a proper subspace again. For the geometry of this work the three have dimension 21 840, 16 384 and 11 151; the middle one is the column rank of $\mathbf{A}$, which is full, so it equals the number of pixels rather than the number of measured detector readings. Only the outermost and the innermost occur in the implementation. $\mathbf{M}$ defines the shape convention just described, and $\mathbf{P}_k = \mathbf{U}_k \mathbf{U}_k^*$ is the projection onto $\text{range}(\mathbf{U}_k)$ that everything else goes through — the simulated noise of Section 3.3, the adversarial perturbation of Section 3.6 and the data-consistency residual of Section 3.7 alike.

The two retained factors are used in different spaces and never interchangeably. $\mathbf{U}_k$ is $m \times k$ and spans the retained part of sinogram space, so everything acting on measurements is built from it: $\mathbf{P}_k$ and the left half of the pseudoinverse. $\mathbf{V}_k$ is $n \times k$ and spans $\text{row}(\mathbf{A})$, the orthogonal complement of the null space in image space, so everything acting on images is built from it: $\mathbf{P}_{\mathcal{N}(\mathbf{A})} = \mathbf{I} - \mathbf{V}_k \mathbf{V}_k^*$, the error decomposition, and the right half of the pseudoinverse. A projector in one space is never the complement of a projector in the other; they are not even of the same size.

3.1 Radon operator

For the implementation of the Radon operator, we used the Astra-Toolbox [12]. It is maintained by the University of Antwerp and supports different kinds of Radon operations. For this experimental setup, we used a 2D parallel-beam Computed Tomography. The first step to obtaining a radon operator in the Astra-Toolbox is done by defining a Volume geometry. It defines the size of the image we want to reconstruct, which in turn determines the size of the emitter and detector array. The second descriptor is the projection geometry, which fixes the beam type (`parallel`), the spacing of the detector elements, their number, and the vector of projection angles. From the two descriptors a projector is built using the `strip` model, which integrates over the strip that a detector element actually sees rather than along an infinitely thin line. This is precisely the discretisation assumed in Section 1.2: a pixel basis on the image side and a finite set of measurements (θ_i, s_i) on the sinogram side.

The geometry used for every experiment in this work is the following.

image resolution	128×128 pixels, so $n = 16\,384$
detector elements	$n_d = 182$
projection angles	$n_\theta = 180$, equispaced over $[0^\circ, 180^\circ)$
limited-angle window	$\varphi = [0^\circ, 120^\circ)$, i.e. 120 of the 180 angles
pixel spacing	$dx = 1$

The detector count is chosen as $\lfloor \sqrt{2} \cdot 128 \rfloor + 1 = 182$, that is, at least the length of the image diagonal, so that no corner of the image can fall outside the detector array for any projection angle.

ASTRA is used to construct the system matrix itself rather than as a matrix-free forward and backprojection pair, because the matrix is what gives access to exact inversion: with its singular value decomposition at hand the pseudoinverse and the null-space projection are closed-form linear operators instead of the output of an iterative solver, and both are ordinary tensor products, which is what lets an attack gradient travel through them. In Python this is implemented in the following way:

```

1 angles = np.linspace(0, 180, 180, endpoint=False)
2 vol_geom = astra.create_vol_geom(128, 128)
3 proj_geom = astra.create_proj_geom('parallel', 1.0, 182, angles)
4 projector_id = astra.create_projector('strip', proj_geom, vol_geom)
5 matrix_id = astra.projector.matrix(projector_id)
6 csr = astra.matrix.get(matrix_id)

```

This creates a projector, callable by its id, that can be used to calculate the radon transform of an image and in the final two lines, calculates a matrix, that can be used to perform the radon operation. The matrix is given as a `scipy.sparse.csr_matrix`, compressed sparse row matrix, and will later be used to calculate the pseudoinverse and the nullspace projections. In the following, let $\mathbf{A}_{\text{full}} := \text{csr}$ be the sparse matrix with dimensions $182 * 180 \times 128 *$

$128 = 32\,760 \times 16\,384$. From $\mathbf{A}_{\text{full}}$ we can construct the limited-angle matrix $\mathbf{A} \in \mathbb{R}^{21\,840 \times 16\,384}$ obtained by keeping only the rows belonging to the 120 measured angles. ASTRA orders the rows angle-major, so row $i \cdot n_d + j$ holds detector j at angle θ_i , $\mathbf{A}$ is obtained by only taking those rows corresponding to measured angles. It is $\mathbf{A}$ that plays the role of $\mathbf{A}$ everywhere in the theory. $\mathbf{A}$ models the limited angle Computed Tomography measurement process the networks aim to invert and the attacks perturb.

Building the matrices and decomposing them is expensive but depends only on the geometry, never on the data. Both are therefore cached on disk under a SHA-256 hash of the resolution, detector count, pixel spacing, angle vector, limited-angle window, truncation threshold and floating-point precision, so that a rerun with the same geometry loads them instead of recomputing them. Every stage of the pipeline — data generation, training and the attacks — asks for the same geometry at the same precision, so one cache entry serves all three and the singular value decomposition is computed exactly once per noise-independent geometry.

3.2 Pseudoinverse

The pseudoinverse of Section 1.2 requires the singular value decomposition of the system matrix, and this is the single most expensive computation in the whole pipeline. $\mathbf{A}$ is densified and decomposed with `torch.linalg.svd` on the GPU. In single precision the dense copy occupies $21\,840 \cdot 16\,384 \cdot 4\text{ B} = 1.4\text{ GB}$. Without a GPU the same dense decomposition runs through LAPACK on the CPU, which is the path the test suite takes. The matrix, the decomposition, the cache on disk and the factors at training and attack time is in single precision. The truncation described in Section 1.2 is applied relative to the largest singular value. A direction is retained if

$$\sigma_i \geq \tau \sigma_{\max}, \quad \tau = 4 \cdot 10^{-3}.$$

Scaling the threshold by $\sigma_{\max}$ rather than fixing it absolutely puts the cutoff in relation to the operator norm, so τ is dimensionless and does not have to be re-tuned when the geometry — and with that the scale of $\mathbf{A}$ — changes. It also controls the conditioning of the retained operator directly, and it is worth spelling that out. A retained direction satisfies $\sigma_i \geq \tau \sigma_{\max}$ by definition, so the smallest retained singular value obeys $\sigma_{\min} \geq \tau \sigma_{\max}$ and the condition number of the retained operator is

$$\frac{\sigma_{\max}}{\sigma_{\min}} \leq \frac{1}{\tau} = 250.$$

Equivalently $\|\mathbf{A}^+\|_2 = 1/\sigma_{\min} \leq 1/(\tau \sigma_{\max})$. Measured relative to the operator norm, the pseudoinverse can amplify by at most $1/\tau$. That is the amplification the truncation exists to limit, and τ sets it directly.

For the geometry of Section 3.1 this keeps 11 151 of 16 384 directions. The

numerical null space $\mathcal{N}(\mathbf{A})$, which is the space the Nullspace Network operates on, has dimension $16\,384 - 11\,151 = 5\,233$, roughly a third of image space. This is not a small correction channel but a substantial subspace, and every statement of Section 1.4 about what the Nullspace Network can and cannot repair refers to this splitting of $\mathbb{R}^{16\,384}$.

Four implementation details are worth recording.

First, the pseudoinverse is never stored by itself. Writing $\mathbf{U}_k, \Sigma_k, \mathbf{V}_k$ for the retained factors, the reconstruction is evaluated as the two matrix products

$$\mathbf{A}^+ \mathbf{y} = \mathbf{V}_k \Sigma_k^{-1} \mathbf{U}_k^* \mathbf{y},$$

applied to a batch of measurement vectors. The reason is that the factors have to be stored regardless, because three further operators are built from them, and once they are in memory an explicit $\mathbf{A}^+$ alongside them would be 1.4 GB of pure duplication.

The two factors earn their keep in different spaces, and it is worth separating them. $\mathbf{V}_k$, which is $n \times k$, carries both image-domain operations: the null-space projector $\mathbf{P}_{\mathcal{N}(\mathbf{A})} = \mathbf{I} - \mathbf{V}_k \mathbf{V}_k^*$ and the error decomposition of Section 1.4, which is the same product. $\mathbf{U}_k$, which is $m \times k$, carries the sinogram-domain ones: the pseudoinverse itself and the projector $\mathbf{P}_k = \mathbf{U}_k \mathbf{U}_k^*$.

Second, the space the Nullspace Network acts in is the numerical null space of the truncated operator, the 5 233-dimensional space counted above, and the space every decomposition figure of Chapter 4 splits along. It too is applied without forming an $n \times n$ matrix. The error decomposition of Section 1.4 is computed the same way: the nullspace-complement component is $\mathbf{e}_{\mathcal{N}^\perp} = \mathbf{V}_k \mathbf{V}_k^* \mathbf{e}$ and the nullspace component is $\mathbf{e}_{\mathcal{N}} = \mathbf{e} - \mathbf{e}_{\mathcal{N}^\perp}$. The two are orthogonal by construction, so $\|\mathbf{e}\|^2 = \|\mathbf{e}_{\mathcal{N}^\perp}\|^2 + \|\mathbf{e}_{\mathcal{N}}\|^2$, which is the identity every decomposition figure in Chapter 4 relies on.

Third, and decisively for this work, all of these operations are ordinary tensor products, so PyTorch differentiates them. The gradient of an attack loss with respect to the sinogram has to travel backwards through the initial reconstruction into the measurements, and it is only because the pseudoinverse is two matrix multiplications that this is possible at all.

Finally, precision and layout. Everything is single precision and dense. Dense, because the sparse kernels are considerably slower for a matrix of this density. The density follows from the geometry rather than from measurement: a row of $\mathbf{A}$ is one ray, and a ray crossing a $\text{res} \times \text{res}$ grid meets on the order of res of its res^2 pixels, so the fraction of nonzero entries is on the order of $1/\text{res}$. At $\text{res} = 128$ that is about 0.8%. A sparse kernel has to pay indirect indexing for every entry it touches and only wins when that buys it a large enough reduction in work; at one per cent of a matrix this size, it does not. Dense is also preferred because the automatic differentiation support for sparse CSR is still experimental, and the attack depends on differentiating through these products.

Single precision is chosen, because it halves both the memory of the dense copy and the size of the cache and does not provide meaningful improvements in accuracy. The obvious worry is the truncation boundary. But a dense singular value decomposition is backward stable, so the computed singular values differ

from the exact ones by at most $p(m, n)\epsilon\sigma_{\max}$ with p a low-order polynomial and ϵ the unit roundoff [13]; in single precision $\epsilon \approx 1.2 \cdot 10^{-7}$, while the cutoff sits at $\tau\sigma_{\max}$ with $\tau = 4 \cdot 10^{-3}$. The boundary is therefore resolved to a relative accuracy of $\epsilon/\tau \approx 3 \cdot 10^{-5}$. The reconstruction inherits the same order of error, since the truncated pseudoinverse amplifies by at most $1/\tau$ relative to the operator norm and ϵ/τ is again the bound. Both are several orders of magnitude below the reconstruction errors of Chapter 4.. This matters during training and not only during the attacks: the Nullspace Network applies $\mathbf{P}_{\mathcal{N}(\mathbf{A})}$ on every forward pass, so it is a per-step cost of training rather than a one-off.

3.3 Data generation

The dataset consists of 5 000 phantoms, each containing a single random ellipse, generated with ODL’s `ellipsoid_phantom` on the image grid supplied by DIVAL’s ellipse dataset. ODL rasterises the ellipse, DIVAL supplies the discretised image space it is rasterised onto. Intensity, semi-axes, centre and rotation are drawn at random, with the semi-axes rejected until the ellipse has an area of at least 0.1 and the centre and rotation rejected until the ellipse lies entirely inside the image. Restricting each image to one ellipse is a deliberate simplification, it guarantees both architectures reach a small clean reconstruction error. This is needed to perform meaningful attacks, since we want to show that the attack can manipulate an otherwise good reconstruction.

It also keeps the null-space component of an error legible by eye. The decomposition figures of Chapter 4 show $\mathbf{e}_{\mathcal{N}}$ as an image, and with one ellipse in the frame that image can be read: the invisible part of a limited-angle error has a characteristic shape — it is elongated along the directions the missing 60° of angles would have measured, and it concentrates at the two edges of the ellipse whose normals point into the missing wedge. With several overlapping objects the same field is a superposition of as many such patterns and shows only that something is wrong, not what. Since the central claim of this work concerns which component of the error an attack grows, being able to look at that component and recognise it is not merely a cosmetic consideration.

For every sample the pipeline performs four steps.

1. Draw the ground truth $\mathbf{x}^\dagger$.
2. Simulate the noise-free measurement $\mathbf{y} = \mathbf{A}\mathbf{x}^\dagger$.
3. Add noise. A standard normal tensor is drawn on the sinogram and projected with $\mathbf{P}_k$, giving $\boldsymbol{\eta}$; it is then rescaled so that the perturbation has a prescribed *relative* size,

$$\mathbf{y}^\delta = \mathbf{y} + \sigma_{\text{rel}} \frac{\|\mathbf{y}\|_2}{\|\boldsymbol{\eta}\|_2} \boldsymbol{\eta}, \quad \text{so that} \quad \|\mathbf{y}^\delta - \mathbf{y}\|_2 = \sigma_{\text{rel}} \|\mathbf{y}\|_2$$

holds exactly and per sample, independently of how bright the phantom is.

4. Compute the initial reconstruction $\mathbf{x}_{\text{init}} = \mathbf{A}^+ \mathbf{y}^\delta$ and store ground truth, sinogram and initial reconstruction to disk.

The projection in step 3 is what the phrase “restricted to the measured angles” has to mean here. Multiplying by $\mathbf{M}$ would not suffice, a component of the measured rows that no image could have produced is not measurement noise but a sinogram no scanner can emit, and $\mathbf{A}^+$ discards it outright, so the nominal σ_{rel} would overstate the perturbation the reconstruction receives.

Projecting onto $\text{range}(\mathbf{U}_k)$ rather than onto the larger $\text{range}(\mathbf{A})$ is the second half of the same argument. The part of a perturbation lying between the two is annihilated by $\mathbf{A}^+$ and so never reaches $\mathbf{x}_{\text{init}}$ either, so drawing the noise there would make the nominal σ_{rel} larger than the perturbation the network is actually trained against. The adversarial budget, in contrast, is spent entirely inside $\text{range}(\mathbf{U}_k)$ and is therefore felt in full. Since the attacks of Section 3.6 set $\varepsilon = \sigma_{\text{rel}}$ precisely in order to grant the attacker as much energy as the noise, any gap between the two would silently break that calibration.

The script `truncation_study.py` measures the gap on the real geometry. Noise drawn in $\text{range}(\mathbf{A})$ delivers 0.822 ± 0.003 of its nominal amplitude to the reconstruction, against 1 when it is drawn in $\text{range}(\mathbf{U}_k)$; an attacker at $\varepsilon = \sigma_{\text{rel}}$ would accordingly have been $1/0.822 = 1.21$ times as strong as the noise it is meant to be indistinguishable from, equally at every noise level. Truncating once, at the same τ , throughout removes that factor — and with it a second decomposition that cost as much as the first, 228s on the A6000, for no other purpose.

Note also that the noise is added after the measurement rather than to the photon counts before it. Physically neither is faithful — real CT noise is Poisson on counts, before the log transform — but the post-measurement Gaussian is the standard idealisation of an inverse problem, and it is what makes the identity above hold exactly and per sample.

Alongside the data the pipeline writes a summary file recording the geometry, the angle vector, the limited-angle window, the noise level, the mean sinogram norm, the mean noise norm, the operator norm and the truncation threshold. Training and attacking both read their configuration from this file rather than from the command line, which guarantees that the operator used to attack a model is the operator that generated the data it was trained on.

Four noise levels are used, $\sigma_{\text{rel}} \in \{0.005, 0.01, 0.02, 0.05\}$, each with its own dataset, its own pair of trained models and its own attack run. Each dataset is split by index, and therefore deterministically, into three parts: the first 3500 samples form the training set, the next 500 the validation set and the last 1000 the test set. The validation set is used for one thing only, choosing which checkpoint to keep, and the test set for nothing but evaluation. Choosing the checkpoint on the test set instead would report every result on the very samples the kept model was selected to do best on, and the clean reconstruction error in particular would come out optimistically biased.

3.4 Residual UNet

To be able to describe the residual UNet, we first need to implement the UNet itself. We start the implementation of the UNet by defining a double convolution:

```
1 class DoubleConv(nn.Module):
2     def __init__(self, in_channels, out_channels, mid_channels=None):
3         super().__init__()
4         mid_channels = mid_channels or out_channels
5         self.net = nn.Sequential(
6             nn.Conv2d(in_channels, mid_channels, 3, padding=1, bias
7             =False),
8             nn.ReLU(inplace=True),
9             nn.Conv2d(mid_channels, out_channels, 3, padding=1,
10            bias=False),
11            nn.ReLU(inplace=True),
12        )
13     def forward(self, x):
14         return self.net(x)
```

Listing 1: DoubleConv

This applies two convolutions: the first takes the channel count to `mid_channels`, the second from there to `out_channels`. Left unspecified the two are equal, so the first convolution sets the channel count and the second leaves it unchanged. The expansive path below is the one place where it is set explicitly, and it is the reason the argument exists at all. In the expansive path, the concatenation doubles the channel count and the double convolution has to bring it down again in two steps rather than one. Kernel size is 3, padding is 1 and there is no bias in these convolutions. Each convolution is followed by a ReLU activation function. The parameter `inplace = true` has the activation function modify the input tensor instead of generating a new output tensor. This decreases memory requirements. We can then use this double convolution, to create a down and an upsampling module.

```
1 class Down(nn.Module):
2     def __init__(self, in_channels, out_channels):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.MaxPool2d(2),
6             DoubleConv(in_channels, out_channels),
7         )
8     def forward(self, x):
9         return self.net(x)
10
11 class Up(nn.Module):
12     def __init__(self, in_channels, out_channels):
13         super().__init__()
14         self.up = nn.Upsample(scale_factor=2, mode="bilinear",
15                               align_corners=True)
16         self.conv = DoubleConv(in_channels, out_channels,
17                                in_channels // 2)
18     def forward(self, x1, x2):
```

```

18     x1 = self.up(x1)
19     diffY, diffX = x2.size(2) - x1.size(2), x2.size(3) - x1.
    size(3)
20     x1 = F.pad(x1, [diffX // 2, diffX - diffX // 2,
21                   diffY // 2, diffY - diffY // 2])
22     return self.conv(torch.cat([x2, x1], dim=1))

```

Listing 2: Down and Up

The downsampling (Down) is just the 2×2 max pooling of Section 1.3, followed by a double convolution which is what raises the channel count. The upsampling (Up) is bilinear upsampling with a scale factor of 2, so it doubles the resolution, and it is the counterpart of that max pooling: pooling keeps one of every four values, bilinear upsampling restores the resolution by interpolating between the values that survived. Bilinear here means linear in each of the two image axes separately. A new pixel falls between four known ones; interpolating linearly along the first axis between each of the two pairs gives two intermediate values, and interpolating linearly between those along the second axis gives the new pixel. The result is a weighted average of the four neighbours whose weights are the products of the one-dimensional distances. The parameter `align_corners = True` fixes the convention for where the known samples sit: the corner pixels of input and output are made to coincide, so the corner values are reproduced exactly and the interpolation grid is stretched to fit between them.

The upsampled tensor is then concatenated with the tensor of the corresponding downsampling step — this is the skip connection of the original U-Net [8]. The concatenation doubles the channel count, and the double convolution it is passed through undoes that doubling: its intermediate channel count is set to half its input, which is exactly the `mid_channels` argument of `DoubleConv`. Should the two tensors disagree in size by a pixel, which happens for odd resolutions, the upsampled one is padded symmetrically before the concatenation; at our resolutions, all powers of two, this never triggers.

A final 1×1 convolution (`OutConv`) reduces the 64 feature channels back to the single output channel. A 1×1 kernel has no spatial extent, so it does not mix neighbouring pixels at all: at each pixel independently it takes the 64 channel values as a vector and returns one weighted sum of them, with bias. It is a learned linear combination across channels, applied identically at every pixel. This collapses a feature stack into one greyscale image. The UNet definition is then the following:

```

1 class UNet(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self.inc = DoubleConv(1, 64)
5         self.down1 = Down(64, 128)
6         self.down2 = Down(128, 256)
7         self.down3 = Down(256, 512)
8         self.down4 = Down(512, 512)
9         self.up1 = Up(1024, 256)
10        self.up2 = Up(512, 128)
11        self.up3 = Up(256, 64)

```

```

12     self.up4 = Up(128, 64)
13     self.outc = OutConv(64, 1)
14
15     def forward(self, x):
16         x1 = self.inc(x)
17         x2 = self.down1(x1)
18         x3 = self.down2(x2)
19         x4 = self.down3(x3)
20         x5 = self.down4(x4)
21         x = self.up1(x5, x4)
22         x = self.up2(x, x3)
23         x = self.up3(x, x2)
24         x = self.up4(x, x1)
25     return self.outc(x)

```

Listing 3: UNET

Four downsampling steps take a 128×128 input down to an 8×8 bottleneck carrying 512 channels, and the expansive path returns it to the original resolution. In total the network has 17 252 993 trainable parameters. The count follows from the shapes alone, since every parameter sits in a convolution kernel: a 3×3 convolution from c_{in} to c_{out} channels without bias holds $9 c_{\text{in}} c_{\text{out}}$ weights, so a `DoubleConv`($c_{\text{in}}, c_{\text{out}}, c_{\text{mid}}$) holds $9(c_{\text{in}} c_{\text{mid}} + c_{\text{mid}} c_{\text{out}})$, and `OutConv` adds $64 \cdot 1 + 1 = 65$ for its kernel and bias. Summing the ten blocks gives

block	channels (in, mid, out)	parameters
inc	1, 64, 64	37 440
down1	64, 128, 128	221 184
down2	128, 256, 256	884 736
down3	256, 512, 512	3 538 944
down4	512, 512, 512	4 718 592
up1	1024, 512, 256	5 898 240
up2	512, 256, 128	1 474 560
up3	256, 128, 64	368 640
up4	128, 64, 64	110 592
outc	$64 \rightarrow 1, 1 \times 1$	65
total		17 252 993

The two innermost blocks `down4` and `up1` hold 10 616 832 of those, about three fifths of the network, which is the usual shape of a U-Net: the resolution is lowest at the bottleneck, so channels are cheapest there in memory and most numerous in parameters. This gives us an image to image neural network, that we can use in our residual and nullspace network architectures. The output of the residual network is then calculated as $\mathbf{x} + \text{UNet}(\mathbf{x})$, which is the reconstruction method $\mathcal{F}_\theta$ of Section 1.3 evaluated at $\mathbf{x} = \mathbf{x}_{\text{init}}$. Let us now talk about how we train the network. The loss function we use is the mean squared error:

```

1 def mse_loss(pred: torch.Tensor, target: torch.Tensor) -> torch.
   Tensor:
2     """training objective: MSE vs ground truth"""
3     return torch.mean((pred - target) ** 2)

```

Listing 4: mse_loss

Where the tensor `pred` is the network reconstruction and `target` is the ground truth. We use an `torch.optim.Adam` optimizer on the model parameters to train each epoch. In PyTorch the optimizer is created as

```
torch.optim.Adam(model.parameters(), lr=0.0001).
```

Before each step we set the gradient to zero, recompute it with `loss.backward()` and update the models parameters using the optimizer accordingly. We train for 50 Epochs with a batch size of 32, without a learning-rate schedule and without data augmentation.

Both are standard devices and both are deliberately omitted. A learning-rate schedule lowers the step size as training proceeds — a large rate early to cross the loss landscape quickly, a small one late to settle into a minimum instead of bouncing around it — and typically buys a somewhat lower final loss. Data augmentation enlarges the training set with label-preserving transformations of it, rotations or flips of the phantom here, and typically buys a smaller generalisation gap. Neither is used because neither would serve the question this work asks. Both architectures are trained by the identical loop, so any accuracy either device bought would be bought for both, while the comparison of the two is what the results rest on; and the epoch study of Section 3.6.3 reads attackability against the epoch index, which a changing learning rate would confound. Adding either is the obvious thing to try if the clean reconstruction error ever became the bottleneck.

Each epoch is followed by an evaluation over the validation set, and whenever the validation loss improves the weights are written to disk. The checkpoint with the lowest validation loss is the one attacked and used throughout.

Two further artifacts are produced during training, both of them for the epoch study of Section 3.6.3. Every N th epoch a snapshot of the weights is written, and at the end of training a history file recording the training and validation loss of each epoch together with the index of the best one.

The stride N is a storage and runtime dial, not an accuracy one. One snapshot of 17.3 million single-precision parameters is 69 MB, so $N = 1$ costs 3.4 GB per model and noise level, 27 GB over the two architectures and four noise levels; and because the epoch study attacks every snapshot it writes, the same factor reappears in the runtime of that study. All experiments in this work use $N = 1$, which is to say the dial is left at its finest setting and every epoch is both stored and attacked. Dropping it provides no benefit, so for ease of reproducibility, it is kept. The history file is what lets the study plot attackability against the loss curves, so it is always written regardless of N . Since the two architectures differ only in the wrapper around the UNet, one training loop serves both.

3.5 Nullspace Network

The Nullspace Network shares the UNet of the previous section exactly, the two architectures differ only in the application of the UNet correction.

```
1 class RESNET(nn.Module):
2     def forward(self, x):
```

```

3         return x + self.unet(x)
4
5     class NSN(nn.Module):
6         def forward(self, x):
7             return x + self.radon.proj_null_image(self.unet(x))

```

Listing 5: The two wrappers

Here `proj_null_image` is the operator $\mathbf{P}_{\mathcal{N}(\mathbf{A})} = \mathbf{I} - \mathbf{V}_k \mathbf{V}_k^*$ of Section 3.2, so the second wrapper is exactly the reconstruction map $\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{init}} + \mathbf{P}_{\mathcal{N}(\mathbf{A})} N_{\theta}(\mathbf{x}_{\text{init}})$ of Section 1.4.

The projection sits *inside* the computational graph and not after it. At inference that is no distinction at all — projecting a trained residual network’s output would give the same data-consistent map — but during training it is one, and the reason is the chain rule. Write $\mathbf{r} = N_{\theta}(\mathbf{x}_{\text{init}})$ for the raw network output, so that the reconstruction is $\mathbf{x}_{\text{out}} = \mathbf{x}_{\text{init}} + \mathbf{P}_{\mathcal{N}(\mathbf{A})} \mathbf{r}$. Differentiating the loss through that last linear step transposes the operator,

$$\nabla_{\mathbf{r}} \mathcal{L} = \mathbf{P}_{\mathcal{N}(\mathbf{A})}^* \nabla_{\mathbf{x}_{\text{out}}} \mathcal{L} = \mathbf{P}_{\mathcal{N}(\mathbf{A})} \nabla_{\mathbf{x}_{\text{out}}} \mathcal{L},$$

the second equality because an orthogonal projector is self-adjoint, $\mathbf{P}^* = \mathbf{P}$. The gradient that arrives at the UNet output is therefore itself a null-space vector: whatever the loss would like to change about the component of $\mathbf{r}$ in $\mathcal{N}(\mathbf{A})^{\perp}$, no gradient signal to that effect ever reaches the weights. This means the network is never rewarded for producing a correction that lies in $\mathcal{N}(\mathbf{A})^{\perp}$, and any such correction will be discarded. The capacity that a post-hoc variant would spend learning such a correction, in our architecture is instead spent inside $\mathcal{N}(\mathbf{A})$.

That the implementation actually realises the properties claimed in Section 1.4 is checked numerically against the real operator, and the precise form of those checks is worth stating, because truncation weakens one of them. The test suite verifies that $\mathbf{P}_{\mathcal{N}(\mathbf{A})}$ is idempotent, that the error decomposition reproduces the error exactly and splits it orthogonally, and that the correction of a Nullspace Network is invisible to the measurements. The data-consistency check is not $\mathbf{A} \mathbf{P}_{\mathcal{N}(\mathbf{A})} = 0$, which under truncation is false: $\mathbf{P}_{\mathcal{N}(\mathbf{A})} = \mathbf{I} - \mathbf{V}_k \mathbf{V}_k^*$ retains the discarded small-singular-value directions, and $\mathbf{A}$ maps those to something small but nonzero. What does vanish exactly, and what the test asserts, is the *measured* trace,

$$\mathbf{P}_k \mathbf{A} \mathbf{P}_{\mathcal{N}(\mathbf{A})} = 0,$$

which is the identity the reconstruction pipeline actually relies on, since that projection is what every measurement in this work passes through. One further point: the check is made with untrained weights and holds for any weights at all, because the property is a property of the architecture and not of what the network has learned.

3.6 Adversarial attacks

In the attack implementation we used, we differentiate the whole chain from the sinogram to the reconstruction,

$$\mathbf{y} \mapsto \mathbf{P}_k \mathbf{y} \mapsto \mathbf{x}_{\text{init}} = \mathbf{A}^+ \mathbf{P}_k \mathbf{y} \mapsto \mathcal{F}_\theta(\mathbf{y}),$$

where $\mathbf{P}_k = \mathbf{U}_k \mathbf{U}_k^*$ is the orthogonal projector onto the retained range of the limited-angle operator. It is emphatically *not* the mask $\mathbf{M}$ of the convention above. Separating the initial reconstruction from the network in the code means that the same attack implementation drives both architectures with no branching inside the attack itself.

The first arrow also applies to the clean sinogram before the attack begins: what is attacked is $\mathbf{P}_k \mathbf{y}^\delta$ rather than the $\mathbf{y}^\delta$ that was written to disk. This changes nothing about the network input, because $\mathbf{A}^+ = \mathbf{V}_k \Sigma_k^{-1} \mathbf{U}_k^*$ annihilates exactly the component the projection removes, so $\mathbf{A}^+ \mathbf{y}^\delta = \mathbf{A}^+ \mathbf{P}_k \mathbf{y}^\delta$ identically; it matters only in that the reported perturbation is measured against the projected sinogram.

Budget. The budget is relative and per sample: $\varepsilon_i = \varepsilon \|\mathbf{y}_i\|_2$, so a bright and a faint sinogram are attacked at the same relative strength. Unless stated otherwise ε is set to σ_{rel} , the relative noise level of Section 3.3 for the dataset the model was trained on, so that the attacker is granted exactly as much energy as the measurement noise the model has already learned to tolerate. A perturbation that does real damage within that budget is a perturbation indistinguishable in size from noise the scanner produces anyway.

Step size and restarts. The step size is derived from the budget, per sample and in the same units,

$$\alpha_i = \frac{2.5 \varepsilon_i}{T}, \quad T = 50,$$

which is the $2.5 \varepsilon / T$ step size customarily used with PGD [10]. The factor 2.5 gives the iteration a total travel of $2.5 \varepsilon_i$, enough to cross the ball from any starting point and still have most of the budget left to refine the direction inside it; a factor near 1 would only just reach the boundary and a much larger one would spend every step being projected back. Tying α_i to ε_i rather than to a dataset-wide mean matters because the budget is itself per sample: a faint sinogram and a bright one are then stepped at the same fraction of their own budget, so the number of effective iterations does not depend on how bright the phantom happens to be.

Each attack is run from 2 independent random starting points. A starting point is drawn as a random direction in $\text{range}(\mathbf{U}_k)$ — $\mathbf{P}_k$ applied to a standard normal vector, which is isotropic within that subspace, normalised to unit length — scaled to a radius drawn uniformly from $[0, \varepsilon_i]$, so that it is a random point of S in the sense of Section 2.1. Merely clipping a Gaussian draw to the ball would not do: its norm is set by the dimension of the sinogram rather than by the budget, so every restart would begin at the same radius and the restarts

would differ only in direction.

Of the restarts, the better one is kept *per sample*, by comparing the objective of each sample separately. Keeping the restart whose objective is higher on average over the batch would hand every sample the same restart, which is the weaker one for any sample whose own best came from the other; the reported attack would then be weaker than the attack actually found, and the models would look more robust than they are.

Feasible set. Writing $\mathbf{P}$ for $\mathbf{P}_k$, the projection Π_S of Section 2.1 is implemented as

$$\Pi_S(\boldsymbol{\delta}) = \mathbf{P} \Pi_{B_\varepsilon}(\mathbf{P}\boldsymbol{\delta}), \quad \Pi_{B_\varepsilon}(\mathbf{v}) = \mathbf{v} \cdot \min\left(1, \frac{\varepsilon}{\|\mathbf{v}\|_2}\right),$$

with the ball projection applied per sample.

The outer $\mathbf{P}$ is, mathematically, redundant. Π_{B_ε} multiplies each sample by a scalar, and scaling a vector cannot take it out of a subspace, so $\Pi_{B_\varepsilon}(\mathbf{P}\boldsymbol{\delta})$ already lies in $\text{range}(\mathbf{A})$ and the second $\mathbf{P}$ changes nothing. It is applied anyway for a numerical reason: the iterate is projected and rescaled once per step for 50 steps in single precision, and without the re-projection the small component that rounding introduces outside the subspace accumulates over those steps. The cost is one additional pair of matrix products per step against an attack that already pays several, and it keeps the invariant the test suite asserts — that the returned $\boldsymbol{\delta}$ satisfies $\mathbf{P}\boldsymbol{\delta} = \boldsymbol{\delta}$ — true to single-precision tolerance rather than approximately.

The feasible set of our attacks is therefore

$$S := \text{range}(\mathbf{U}_k) \cap B_\varepsilon,$$

the same set the simulated measurement noise of Section 3.3 is drawn from. That the two coincide is what makes the choice $\varepsilon = \sigma_{\text{rel}}$ mean what it is supposed to mean: attacker and scanner are given the same amount of energy in the same subspace, so a perturbation that does damage within the budget really is one that could hide among the noise.

Restricting the attacker to $\text{range}(\mathbf{U}_k)$ rather than to $\text{range}(\mathbf{A})$ costs nothing in strength. Energy placed in the discarded directions is suppressed by $\mathbf{A}^+$ before it reaches the network, so it could only be budget the attacker wastes.

With these pieces the iteration of Section 2.1 is ten lines of code. In the listing below, `project_delta` is the projection Π_S given above, and `adapter.projector` is $\mathbf{P}$.

```

1 for _ in range(SUITE_RESTARTS):
2     delta = adapter.projector(proj_l2_ball(torch.randn_like(y_clean)
3     ), eps))
4     delta = project_delta(delta, eps, adapter.projector)
5
6     for _ in range(SUITE_STEPS):
7         y_adv = (y_clean + delta).detach().requires_grad_(True)
8         pred, _, _ = adapter.forward(y_adv, project=False)

```

```

8     grad = torch.autograd.grad(loss_of(pred), y_adv)[0]
9     with torch.no_grad():
10         g = grad / l2_norm_batch(grad).clamp_min(1e-12).view
            (-1, 1, 1, 1)
11         delta = project_delta(delta + alpha * g, eps, adapter.
            projector)

```

Listing 6: The PGD loop

Three details of the listing correspond to the derivation. The gradient is taken with respect to the perturbed sinogram y_{adv} rather than the perturbation, which is the same gradient because the two differ by the constant y , and it is the form autograd needs. PyTorch records each operation as a node of a graph whose *leaves* are the tensors it was told to differentiate with respect to; gradients are accumulated at the leaves and nowhere else. Detaching y_{adv} and marking it `requires_grad` makes it such a leaf, which is why the loop rebuilds it each step rather than carrying `delta` through the graph. The step is *added*, so that the loss is maximised. And the gradient is normalised in the ℓ_2 norm before the step, exactly as derived in Section 2.1. The model is held in evaluation mode throughout and gradients are requested only with respect to the input, so no weight is ever modified by an attack.

Five attacks are run over every model, differing only in the loss that `loss_of` evaluates: the untargeted attack on the total error, the two subspace attacks of the next section, and the two targeted attacks of the one after that.

3.6.1 Targeted Adversarial Attacks on Subspaces

The two subspace losses of Section 2.1.1 are direct transcriptions of their definitions.

```

1 if objective == "null":
2     err = pred - x_gt
3     return reduce_loss(radon.proj_null_image(err) ** 2)
4
5 if objective == "range":
6     err = pred - x_gt
7     return reduce_loss((err - radon.proj_null_image(err)) ** 2)

```

Listing 7: The subspace objectives

Up to the constant factor by which a mean differs from a squared norm, the first is $\mathcal{L}_{\mathcal{N}}$ and the second $\mathcal{L}_{\mathcal{N}^\perp}$. The constant factor of course does not affect the direction of the gradient. Both use the same projector $\mathbf{P}_{\mathcal{N}(\mathbf{A})}$ as the Nullspace Network itself, and for an attack on that network it therefore occurs twice in the graph — once inside the model, where it constrains the correction, and once inside the loss, where it selects the component being maximised — and is differentiated both times. For an attack on the residual network it occurs only in the loss.

Applying it once costs two matrix products against $\mathbf{V}_k$, namely $\mathbf{V}_k^* \mathbf{v}$ followed by $\mathbf{V}_k(\cdot)$, that is $2kn \approx 3.7 \cdot 10^8$ multiply-accumulates per sample, where $k = 11\,151$ is the number of retained singular directions and $n = 16\,384$ the number of

pixels. Against the Nullspace Network the attack pays that four times per step — twice in the forward pass and twice again in the backward pass — for 50 iterations and 2 restarts on every sample. It remains affordable because these are dense single-precision products on the GPU, which is what the layout choice of Section 3.2 was made for.

3.6.2 Targeted Adversarial Attacks using Examples

The two targeted losses of Section 2.1.2 are negated squared distances, so that maximising them moves the reconstruction towards the target.

```

1 if objective == "zero":
2     return -reduce_loss(pred ** 2)
3
4 if objective == "target":
5     return -reduce_loss((pred - target.detach()) ** 2)

```

Listing 8: The targeted objectives

The first realises $t = 0$ and needs no reference image at all. The second does, and it is supplied once per batch: the target of each sample is the ground truth of a different sample of the same batch, so no extra data is needed and the targets are as plausible as the phantoms themselves.

The targets are assigned by a derangement of the batch indices — a permutation that leaves no index in place. It is drawn by rejection: a random permutation is sampled and redrawn if it fixes any index, with a cyclic shift used as a fallback in the rare case that twenty draws all fail. The point is that no sample may be steered towards its own ground truth, which would turn the targeted attack into no attack at all for that sample and would quietly weaken the reported numbers.

The target enters the loss as `target.detach()`. Detaching a tensor returns a view of it that is cut out of the computational graph, so no gradient can flow back through it. Here that is defensive rather than necessary: the target is ground truth read from disk and carries no graph to begin with. It states that the target is a fixed reference and not a quantity the attack may optimise, and it keeps that true if the target were ever computed from something differentiable.

3.6.3 Epoch study

The epoch study investigates, whether there is a specific time during training, when a model becomes attackable. This is helpful to decide, whether attackability coincides with the onset of overfitting. It consumes the epoch snapshots written by the training loop of Section 3.4. For each architecture, every snapshot is loaded in turn, attacked with the untargeted squared-error objective at a fixed budget of $\varepsilon_{\text{rel}} = 0.01$, and evaluated. The study writes its own table, one row per snapshot, carrying the adversarial error of that snapshot next to the training and validation loss which it reads back from the history file of Section 3.4. The two quantities therefore end up on one axis without the training loop and the attack having to know anything about each other. Because the study

repeats a full attack once per snapshot, it runs on 32 test samples rather than the 128 used for the other attacks.

The inputs are cached once and every snapshot is attacked on that same cache. This matters for reading the resulting curve: the phantoms, their sinograms and their initial reconstructions are byte-identical from epoch to epoch, so the only quantity that varies along the curve is the weights, and any trend in it is a trend in the model rather than in the sample of test data it was measured on.

3.7 Metrics and error decomposition

Every attacked sample is scored with the same set of seven metrics, each computed between a reconstruction $\hat{\mathbf{x}}$ and the ground truth $\mathbf{x}^\dagger$. No single one of them is sufficient, which is why all seven are kept: they disagree about what counts as damage, and an attack that looks mild under one can be severe under another.

- **Relative ℓ_2 error**, $\|\hat{\mathbf{x}} - \mathbf{x}^\dagger\|_2 / \|\mathbf{x}^\dagger\|_2$. The headline number of this work, because its square is the quantity the error decomposition below is additive in — the numerator splits as $\|\mathbf{e}\|^2 = \|\mathbf{e}_{\mathcal{N}^\perp}\|^2 + \|\mathbf{e}_{\mathcal{N}}\|^2$ and the denominator is common to both terms. Normalising by $\|\mathbf{x}^\dagger\|_2$ makes it comparable across phantoms of different brightness; the denominator is floored, since a phantom whose ellipse falls below the sampling grid is all zero and would otherwise blow the ratio up.
- **RMSE**, $\sqrt{\text{mean}((\hat{\mathbf{x}} - \mathbf{x}^\dagger)^2)}$, the same error unnormalised, in image units. Kept because the relative error hides how bright an artefact actually is.
- **NRMSE**, the RMSE divided by the data range $\max \mathbf{x}^\dagger - \min \mathbf{x}^\dagger$. A second normalisation, by contrast rather than by energy.
- **PSNR**, $20 \log_{10}(\text{range}) - 10 \log_{10}(\text{MSE})$ in decibels. The same information as the RMSE on a logarithmic scale, reported because it is the conventional currency of the reconstruction literature.
- **SSIM**, structural similarity. The only metric here that is not a function of the pixelwise difference alone: it compares local means, variances and covariances, so it responds to structure being moved or invented rather than merely to intensity being wrong. That is exactly the failure mode a targeted attack aims for, and the one a pure ℓ_2 figure understates.
- **MAE**, $\text{mean}|\hat{\mathbf{x}} - \mathbf{x}^\dagger|$. Less sensitive to a few large deviations than the squared metrics, so a gap between MAE and RMSE indicates that the error is concentrated rather than spread.
- **Maximum absolute error**, $\max|\hat{\mathbf{x}} - \mathbf{x}^\dagger|$. The opposite extreme: it sees only the single worst pixel. A small mean error with a large maximum is a localised artefact, which is the clinically relevant kind.

All seven are recorded for the clean and for the adversarial reconstruction, and separately for the initial reconstruction, so that the damage the attack does to the network input can be told apart from the damage it does to the output. Aggregation reports the mean with a 95% confidence interval of half-width $1.96 s/\sqrt{N}$, where s is the sample standard deviation of the metric over the N attacked samples and 1.96 the normal quantile, together with the median and the quartiles. The interval is reported because the headline comparison is a comparison of two means over $N = 128$ samples, and a difference smaller than the two intervals is not a difference at all; it quantifies the sampling error of this particular test set, not the variability of the attack. The median and quartiles are reported alongside because the distribution over samples is skewed — a handful of samples on which an attack succeeds unusually well would otherwise dominate the mean — so a gap between mean and median is itself informative. In addition, every error field is decomposed as in Section 3.2 into $\mathbf{e}_{\mathcal{N}}$ and $\mathbf{e}_{\mathcal{N}^\perp}$, and the norms of both components as well as their fractions of $\|\mathbf{e}\|$ are recorded. Since only the squared ℓ_2 norm is additive over this decomposition, the remaining metrics cannot be split the same way. Instead the component-only images $\mathbf{x}^\dagger + \mathbf{e}_{\mathcal{N}^\perp}$ and $\mathbf{x}^\dagger + \mathbf{e}_{\mathcal{N}}$ are scored with the same metrics, which shows how much of the loss in, say, SSIM each subspace is responsible for on its own. The construction deserves a word of caution: these are not reconstructions any method produced, they are the ground truth with one component of an error added back, and the two numbers do not add up to the number for the full reconstruction, because SSIM and PSNR are not additive over the decomposition — that is precisely why the construction is needed. They are to be read as an attribution of the damage, not as a decomposition of the score. Data consistency is measured as the relative residual of the reconstruction against the measurement it should explain,

$$\frac{\|\mathbf{P}_k \mathbf{A} \hat{\mathbf{x}} - \mathbf{y}\|_2}{\|\mathbf{y}\|_2}.$$

Three choices in that formula are deliberate. The forward operator is the sparse $\mathbf{A}$ itself and not the truncated factorisation $\mathbf{U}_k \Sigma_k \mathbf{V}_k^*$, which is correct rather than merely convenient: truncation is a property of the *inverse*, introduced to bound noise amplification, and there is no corresponding notion for the forward direction — a scanner measures what it measures. Using the truncated factorisation here would discard whatever part of $\hat{\mathbf{x}}$ lies in the sub-threshold directions *before* comparing, which would make a reconstruction look consistent precisely in the components the measurement cannot pin down. The operator is also the limited-angle $\mathbf{A}$ rather than the full one: the unmeasured angles carry no data to be consistent with, so including them would score the reconstruction against numbers no scanner produced. And $\mathbf{P}_k$ is applied to $\mathbf{A} \hat{\mathbf{x}}$ rather than omitted, because it is not redundant: $\hat{\mathbf{x}}$ is an arbitrary image, so $\mathbf{A} \hat{\mathbf{x}}$ may have a component in the discarded small-singular-value directions, whereas $\mathbf{y}$ by construction does not. Leaving that component in would add to the residual a term that measures nothing about data consistency. With $\mathbf{P}_k$ applied, both sides of the difference lie in $\text{range}(\mathbf{U}_k)$ — the one subspace the noise, the reconstruction

and the attack all live in.

Two versions of this number are recorded and they mean quite different things. Evaluated against the *attacked* sinogram, the Nullspace Network drives it to about 10^{-8} whatever the attack does — that is the guarantee of Section 1.4 restated, a tautology rather than a robustness result. Evaluated against the *clean* sinogram it reports how far the attack has pushed the reconstruction away from the measurements actually taken and it is this version that carries information. Finally, an attack-independent robustness measure is computed. Every number above is the outcome of one particular attack, so none of them can distinguish a robust model from one the attack merely failed to find a direction for. Write $f : \mathbf{X} \rightarrow \mathbf{X}$ for the image-to-image map the wrapper implements, so $f(\mathbf{x}) = \mathbf{x} + N_\theta(\mathbf{x})$ for the residual network and $f(\mathbf{x}) = \mathbf{x} + \mathbf{P}_{\mathcal{N}(\mathbf{A})}N_\theta(\mathbf{x})$ for the Nullspace Network, and let $g(\mathbf{x}) = f(\mathbf{x}) - \mathbf{x}$ be the learned correction alone. Linearising g around the clean initial reconstruction $\mathbf{x}_0$ and restricting both its input and its output to $\mathcal{N}(\mathbf{A})$ with the same projector gives

$$\mathbf{T} = \mathbf{P}_{\mathcal{N}(\mathbf{A})} \mathbf{J}_g(\mathbf{x}_0) \mathbf{P}_{\mathcal{N}(\mathbf{A})},$$

whose largest singular value $\sigma_{\max}(\mathbf{T})$ is the local gain of the learned channel: the factor by which a worst-case null-space perturbation of the input can be amplified into null-space error at the output. A value below one means the correction contracts in that subspace, above one that it amplifies.

It is estimated by power iteration, and the Jacobian is never assembled. One power-iteration step needs $\mathbf{T}\mathbf{d}$ and $\mathbf{T}^*\mathbf{u}$, which are exactly a Jacobian-vector and a vector-Jacobian product of g with a projection on each side, and PyTorch supplies both directly:

```

1 def G(x):                                     # learned correction, output
2     restricted to N(A)
3     return proj(model(x) - x)
4
5 d = proj(torch.randn_like(x0))
6 d = d / torch.linalg.norm(d.reshape(-1))
7 for _ in range(LIPSCHITZ_ITERS):
8     _, u = torch.autograd.functional.jvp(G, x0, d, strict=False)
9     _, w = torch.autograd.functional.vjp(G, x0, proj(u), strict=False)
10    w = proj(w)
11    d = w / torch.linalg.norm(w.reshape(-1))
12 _, u = torch.autograd.functional.jvp(G, x0, d, strict=False)
13 sigma = torch.linalg.norm(proj(u).reshape(-1))

```

Listing 9: One power iteration of the null-restricted gain

Two features distinguish this from a plain Lipschitz estimate of the network. It is *local*: it is the gain of the linearisation at one point $\mathbf{x}_0$, not a global bound over all inputs, so it describes the neighbourhood the reconstruction actually lives in rather than a worst case over image space — which is what makes it comparable to an attack that also starts there. And it is *restricted*: input and output are both confined to $\mathcal{N}(\mathbf{A})$. That restriction is what makes the number comparable between the two architectures at all. Unrestricted, the Nullspace Network’s

correction is identically zero on $\mathcal{N}(\mathbf{A})^\perp$ while the residual network’s is not, so an unrestricted estimate would mostly measure the architectural constraint rather than what either network has learned. Measured on the subspace both are free to act in, the comparison is between the learned maps. The estimate is averaged over several clean initial reconstructions, and the maximum over them is reported alongside the mean.

3.8 Experimental pipeline

Computation and rendering are kept strictly apart. An attack run writes only numeric artifacts: the per-sample metrics as a table, the adversarial sinograms and perturbations, the example images with their per-sample metrics, the cross-model transfer stacks and the aggregated summary. Every figure in Chapter 4 is produced afterwards from those files alone, without a model, a checkpoint or a Radon operator being loaded. A figure can therefore be redrawn at any time without repeating the attack that produced it, which matters because a full attack run takes several GPU-hours.

The four noise levels are processed by a Slurm pipeline as four job arrays over three dependency levels: data generation and training first; then the attack suite and the epoch study, which both depend only on that first level and therefore run concurrently; then the rendering, which waits for both. Because the four noise levels are entirely independent of one another, each level of the pipeline is an array over them rather than a loop, so the critical path is one noise level deep rather than four.

Reproducibility rests on three things. The random seeds are fixed for data generation, for training and for the attacks. Training is reproducible closely rather than bit for bit, because the convolution kernels cuDNN selects on the GPU are not deterministic. All models within a run are attacked on a cached set of inputs, so that they see identical data and any difference between them is a difference of architecture. And the operator identities the argument of this thesis depends on — the idempotence of the null-space projector, the vanishing of its measured trace $\mathbf{P}_k \mathbf{A} \mathbf{P}_{\mathcal{N}(\mathbf{A})} = 0$, the agreement of the truncated factors with the sparse system matrix, the orthogonality of the error decomposition and the enforcement of the attack budget — are covered by a test suite that exercises them against the real operator rather than a mock, as are the per-sample choice of restart and the integrity of the geometry cache.

4 Results

4.1 Model performance

4.1.1 Reconstruction accuracy

4.1.2 Dataconsistency

4.2 General adversarial attack

4.3 Targeted adversarial attack

4.4 Epoch study

5 Conclusion and Outlook

5.1 Conclusion

5.2 Outlook

References

- [1] Reda R. Gharieb. *Computed-Tomography (CT) Scan*. IntechOpen, 2022.
- [2] Hanming Zhang et al. “Image Prediction for Limited-angle Tomography via Deep Learning with Convolutional Neural Network”. In: (2016).
- [3] Vegard Antun et al. “On instabilities of deep learning in image reconstruction and the potential costs of AI”. In: (2020).
- [4] Thomas Flohr. “CT Systems”. In: *Current Radiology Reports* (2013). DOI: 10.1007/s40134-012-0005-5.
- [5] Eran Malach and Shai Shalev-Shwartz. “Is Deeper Better only when Shallow is Good?” In: (2019).
- [6] R. K. Srivastava, K. Greff, and J. Schmidhuber. “Highway networks”. In: (2015).
- [7] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: (2015).
- [8] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: (2015).
- [9] Han Xu et al. “Adversarial Attacks and Defenses in Images, Graphs and Text: A Review”. In: *International Journal of Automation and Computing* (2020). DOI: 10.1007/s11633-019-1211-x.
- [10] Aleksandr Madry et al. “Towards Deep Learning Models Resistant to Adversarial Attacks”. In: (2019).
- [11] Pytorch, the linux foundation. *Pytorch documentation*. 2026. URL: <https://docs.pytorch.org/docs/2.14/index.html> (visited on 09/05/2026).

- [12] imec-Vision Lab, University of Antwerp. *ASTRA Toolbox*. 2010. URL: <https://astra-toolbox.com/> (visited on 08/08/2026).
- [13] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. 4th ed. Baltimore: Johns Hopkins University Press, 2013. ISBN: 9781421407944.